

Sistemas Operativos

2º Parcial 2C2024 – TM – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

1.

	Fragmentación	Multiprogramación	Tamaño máximo	Overhead
Particiones Fijas	Sufre únicamente fragmentación interna.	Limitado por la cantidad de particiones.	Limitado por la partición más grande.	Más bajo al tener estructuras fijas.
Particiones Dinámicas	Sufre únicamente fragmentación externa.	No se ve limitado.	No se ve limitado.	Más alto debido a la complejidad de sus estructuras y compactación.

2. La DL 8101FFFFh hace referencia a un byte mayor que 2^{30} (1 GiB), por lo tanto aún si fuera el único proceso en Memoria, el mismo no podría ser cargado por completo sin Memoria Virtual. Podría ocurrir si el mismo utilizara la técnica de *overlaying* para que solo una porción del proceso sea cargada en Memoria.

3.

Tabla archivos abiertos P1: FCB1, modo: lectura, puntero R/W: 0 Tabla archivos abiertos P2: FCB1, modo: lectura, puntero R/W: 0	Tabla global archivos abiertos: FCB1, aperturas: 2
--	---

4. Deben borrarse 2 entradas de directorio (las de los hard links) y liberar el inodo correspondiente al archivo. No tendría efecto alguno borrar el soft link ya que es un archivo independiente (con su propio inodo).
5. a) Falso, el mismo es indispensable para saber qué páginas deben ser actualizadas en disco al ser reemplazadas, independientemente del algoritmo de reemplazo.
b) Falso, también sufre fragmentación interna por estar dividido en bloques, pudiendo existir en el último bloque de cada archivo.

Práctica

1. a) El estado de la memoria propuesto puede tener tantas soluciones como personas resolviendo el examen. Una propuesta posible es:

P0, Tamaño: 500 MiB
Libre: 100 MiB
P1, Tamaño 100 MiB
Libre: 100 MiB
P2, Tamaño: 224MiB

b) punteros x bloque = tam_bloque / tam_puntero
 $= 2^{10} \text{ B} / 2^2 \text{ B} = 2^8 = 256$
 $\text{tam_max_archivo} = 2^8 * 2^{10} \text{ B} = 2^{18} \text{ B} = \mathbf{256 \text{ KiB}}$

1 => No se podría tener un archivo con el tamaño de ninguna de estas particiones, en esta propuesta la más chica es 100 MiB > 256 KiB.

$$2^x \text{ B} / 2^2 \text{ B} * 2^x \text{ B} = 1 \text{ GiB}$$

$$2^{(2x-2)} \text{ B} = 2^{30} \text{ B}$$

$$2x - 2 = 30 \rightarrow x = 16$$

2 => Aumentando el tamaño de bloque a **64 KiB** podría tener archivos lo suficientemente grandes:

$$2^{14} * 2^{16} \text{ B} = 2^{30} \text{ B} = 1 \text{ GiB}$$

2. Del proceso B tenemos presentes las páginas 0, 2 y 13.

Del hexa **20CAh**, según cómo lo dividamos podemos formar las páginas 0 y 2.

Como es la última referencia del proceso sería la página 2 dado que el bit de uso está en 1 y su TUR es el más grande.

La página 0 se descarta porque su TUR es más chico que el de la página 2.

Entonces de los 16 bits para direccionar, la DL utiliza 4 bits para número de página y 12 bits para desplazamiento.

Referencia	DL=PAG/DESP
PA A015h (E)	10/015
PB D122h (E)	13/122
PC 2333 h (L)	2/333
PB 5111h (E)	5/111

Algoritmo Clock Modificado					
Marco	TP-PAG/U/M	PA-10 (E)	PB-13 (E)	PC-2(L)	PB-5 (E)
0	PC-5/1/1	PC-5/1/1**	PC-5/1/1**	PC-5/1/1**	PC-5/0/1
1	PC-2/0/0	PC-2/0/0	PC-2/0/0	PC-2/1/0	PC-2/0/0
2	PA-2/1/1	PA-2/1/1	PA-2/1/1	PA-2/1/1	PA-2/0/1
3 **	PB-13/1/0	PB-13/1/0	PB-13/1/1	PB-13/1/1	PB-13/0/1
4	PA-0/0/1	PA-0/0/1	PA-0/0/1	PA-0/0/1	PB-5/1/1
5	PA-7/1/0	PA-7/1/0	PA-7/1/0	PA-7/1/0	PA-7/1/0**
6	PC-7/1/1	PC-7/1/1	PC-7/1/1	PC-7/1/1	PC-7/1/1
7	PB-2/1/1	PB-2/1/1	PB-2/1/1	PB-2/1/1	PB-2/1/1
8	PB-0/0/0	PA-10/1/1	PA-10/1/1	PA-10/1/1	PA-10/1/1
PF		1	0	0	1
Accesos Disco		1	0	0	2
Escritura Disco		0	0	0	1
DF		PA - 8015h	PB - 3122h	PC - 1333h	PB - 4111h

Algoritmo LRU					
Marco	TP-PAG/M/TUR	PA-10 (E)	PB-13 (E)	PC-2(L)	PB-5 (E)
0	PC-5/1/110	PC-5/1/110	PC-5/1/110	PC-5/1/110	PC-5/1/110
1	PC-2/0/115	PC-2/0/115	PC-2/0/115	PC-2/0/130	PC-2/0/130
2	PA-2/1/95	PA-2/1/95	PA-2/1/95	PA-2/1/95	PA-2/1/95
3	PB-13/0/85	PB-13/0/85	PB-13/1/125	PB-13/1/125	PB-13/1/125
4	PA-0/1/70	PA-0/1/70	PA-0/1/70	PA-0/1/70	PA-0/1/70
5	PA-7/0/65	PA-7/0/65	PA-7/0/65	PA-7/0/65	PA-7/0/65
6	PC-7/1/45	PA-10/1/120	PA-10/1/120	PA-10/1/120	PA-10/1/120
7	PB-2/1/100	PB-2/1/100	PB-2/1/100	PB-2/1/100	PB-2/1/100
8	PB-0/0/60	PB-0/0/60	PB-0/0/60	PB-0/0/60	PB-5/1/135
PF		1	0	0	1
Acceso Disco		2	0	0	1
Escritura Disco		1	0	0	0
DF		PA - 6015h	PB - 3122h	PC - 1333h	PB - 8111h

3. a) Suponiendo que cada archivo regular puede estar vacío (tamaño = 0, ocupando 0 bloques), la limitación viene por la cantidad de entradas de directorio que puede haber en nuestro único directorio.

$$\text{tam_directorio} / \text{tam_entrada_directorio} = 16 \text{ MiB} / 32 \text{ B}$$

$$= 2^{24} \text{ B} / 2^5 \text{ B}$$

$$= 2^{19} = 524288 \text{ archivos regulares}$$

Si, en cambio, suponemos que cada archivo debe tener al menos un bloque asignado (tamaño > 0), se debería analizar también como limitación la cantidad de bloques libres del FS.

b) Analicemos el tamaño del FS:

Según la entrada de directorio, los punteros a clusters son de 2 bytes = 16 bits.

$$\text{Tam_fs} = 2^{16} * \text{tam_bloque}$$

$$= 2^{16} * 512 \text{ B}$$

$$= 2^{16} * 2^9 \text{ B} = 2^{25} \text{ B} = 32 \text{ MiB (más chico que el disco)}$$

Como en el FS ya existe un archivo (el directorio raíz) que ocupa 16 MiB, un nuevo archivo podría ocupar como máximo los **16MiB** de espacio libre que queda.

$$\text{c) tam_fat} = 2^{16} * \text{tam_puntero}$$

$$= 2^{16} * 2 \text{ B} = 2^{17} \text{ B} = 128 \text{ KiB}$$